

MARTA Pulse — Execution Guide v2 (Rebuild)

Corrected end-to-end build order incorporating every lesson from build #1 (full detail in LESSONS.md). Amber boxes mark the places the first build went wrong. Library v0.2.0: rail timestamps are now normalized to UTC at the source. Infra now provisions Flex Consumption (FC1) instead of the EOL Linux Consumption plan.

Phase 0 — Prerequisites

1. Azure subscription; Fabric capacity (F2+); ADO project (bobitservices/marta-pulse) with a workload-identity service connection.
2. **Check App Service quota now:** portal → Quotas → provider **App Service** → your region → **Flex Consumption** instances ≥ 1 (the infra now provisions FC1, not the EOL Linux Consumption Y1 — Flex has its own quota, separate from Dynamic SKU). Fresh subscriptions ship with 0 and the Function deploy fails.
3. MARTA rail API key (itsmarta.com signup). Bus feeds need the FULL .pb URLs — the developer page's link text is truncated:

VP: <https://gtfs-rt.itsmarta.com/TMGTFSTimeWebService/vehicle/vehiclepositions.pb>
TU: <https://gtfs-rt.itsmarta.com/TMGTFSTimeWebService/tripupdate/tripupdates.pb>
GTFS static: https://itsmarta.com/google_transit_feed/google_transit.zip

4. Python 3.11+, Azure CLI, Azure Functions Core Tools.

■ **Lesson:** Clone the repo OUTSIDE OneDrive (e.g. C:\dev\marta-pulse). OneDrive file locking corrupted the git index and served stale reads in build #1.

Phase 1 — Local setup

```
git clone https://dev.azure.com/bobitservices/marta-pulse/_git/marta-pulse C:\dev\marta-pulse
cd C:\dev\marta-pulse
python -m venv .venv && .venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]" # tzdata included for Windows zoneinfo
pytest -q # expect: 23 passed
python -m pip install build && python -m build --wheel
# -> dist/marta_pulse-0.2.0-py3-none-any.whl
```

■ **Lesson:** Always python -m pip inside the venv (bare pip.exe was blocked). Bump the wheel version on every library change — identical filenames made 'did the new wheel take?' undiagnosable in Fabric Environments.

Phase 2 — Fabric dev workspace

Create workspace MartaPulse-dev, then:

1. **Lakehouse** MartaPulse_LH, schemas enabled. Immediately run in a notebook: CREATE SCHEMA IF NOT EXISTS bronze/silver/gold (schema-enabled lakehouses don't auto-create them).
2. **Eventhouse** + KQL DB; run DatabaseSchema.kql (creates raw_events, mapping canonical_v1, materialized views).

3. **Eventstream** — a **STANDARD** eventstream. Do NOT create a schema-set/schema-enforced stream.

■ **Lesson:** Schema-enforced Eventstreams expect CloudEvents envelopes — plain JSON from the Function is silently DROPPED ('CloudEvent property type is missing'), and the Lakehouse destination disappears from the menu. The contract is enforced by the KQL mapping + Silver DQ instead.

4. Add **Custom endpoint** source (protocol: **Event Hub** — matches the Function's SDK). Copy the connection string; the EntityPath=es_..._eh suffix is the entity name for Bicep.

5. **Single destination** (add after events flow, in Phase 3): Eventhouse via **Direct ingestion** → raw_events + mapping canonical_v1 — never the Map-schema UI. Then enable **OneLake availability** on raw_events and create a Lakehouse **shortcut** to it in the bronze schema, keeping the name raw_events. No second Eventstream destination.

■ **Lesson:** Revised architecture: one write path. Two destinations doubled storage and CU on an F2 that was already throttling ('Large capacity delays'). The KQL schema types every column (event_ts is datetime — normalized at the source since v0.2.0), so the mirrored Delta arrives typed for free. Mirror latency is ~5–10 min; Silver's lookback window absorbs it.

6. **Environment** MartaPulse_ENV: upload the 0.2.0 wheel, publish, set as workspace default (covers pipeline-triggered notebook runs).

Phase 3 — Azure infra + Function

```
az group create -n rg-martapulse-dev -l eastus
az deployment group create -g rg-martapulse-dev -f infra/main.bicep `
  -p env=dev busVpUrl="<VP .pb url>" busTuUrl="<TU .pb url>" `
  eventstreamEntityName="es_..._eh"
# forward slashes in -f path; connection string is NOT a template parameter
# provisions FC1 Flex Consumption + identity-based storage + deploy container
```

Grant yourself secret rights, then set both secrets (full connection string incl. EntityPath):

```
az role assignment create --assignee <you> --role "Key Vault Secrets Officer" `
  --scope $(az keyvault show -n kv-martapulse-dev --query id -o tsv)
# wait 2-5 min for RBAC propagation
az keyvault secret set --vault-name kv-martapulse-dev -n rail-api-key --value "..."
az keyvault secret set --vault-name kv-martapulse-dev -n eventstream-connection `
  --value "Endpoint=sb://...;...;EntityPath=es_..._eh"

copy dist\marta_pulse-0.2.0-py3-none-any.whl functions\ingest_gtfs_rt\
Add-Content functions\ingest_gtfs_rt\requirements.txt "./marta_pulse-0.2.0-py3-none-any.whl"
cd functions\ingest_gtfs_rt
func azure functionapp publish func-martapulse-dev --python
```

■ **Lesson:** --python is required (language detection fails otherwise). Secrets are read at startup — az functionapp restart after changing them. Verify via App Insights: union traces, exceptions | order by timestamp desc. Expect 'ingested N canonical events' every 15s.

Now wire the two Eventstream destinations (Phase 2.5) with live data in the preview, and confirm rows in raw_events and bronze.telemetry_events.

Phase 4 — Git integration

1. Feature branch → Workspace settings → Git integration → ADO repo, branch, folder fabric/.

2. Initial sync direction: **workspace** → **Git** (never Git → workspace with live items — it overrides/deletes them).

3. **Commit every UI-authored item immediately**, starting with the Lakehouse — Git is your restore point.

■ **Lesson:** Build #1 lost the Lakehouse (and its data): it was never committed, and an Undo reverted the workspace to a Git state without it. Also: items in Git but not the workspace appear as staged DELETIONS — use Undo to import them, never commit those rows. Don't hand-author .platform files (logicalId conflicts block commits), and keep foreign files out of item folders.

4. Pull the notebooks/pipeline into the workspace (Undo on the deletion rows, or Updates tab). Re-attach each notebook's default lakehouse; in the pipeline, re-select both notebooks and set the HTTP connection. Leave activity Connection fields EMPTY — never paste @activity() expressions there.

Phase 5 — Batch spine

1. Pipeline Copy activity: HTTP connection base `https://itsmart.a.com`, relative URL `google_transit_feed/google_transit.zip`, sink `Files/bronze/gtfs_static/incoming`.

2. Run `PL_GTFS_Static_Ingest` → Bronze tables + registry → SCD2 Silver dims.

```
-- SQL endpoint is T-SQL: booleans are 1/0, not true/false
SELECT feed_version, COUNT(*) FROM silver.dim_trip
WHERE is_current = 1 GROUP BY feed_version; -- exactly one row
```

■ **Lesson:** If a first write ever fails mid-create, the catalog can hold an orphan entry (DELTA_TABLE_NOT_FOUND on append). `DROP TABLE` + `mssparkutils.fs.rm` the `Tables//` folder, then rerun.

Phase 6 — Stream conformance + Gold

1. `NB_Silver_Telemetry_Stream` — **batch-incremental** over the `bronze.raw_events` shortcut (Structured Streaming can't read Kusto-mirrored Delta; the service compacts files). Ingest-time window + lookback + anti-join on `event_id` = idempotent. Runs from `PL_Telemetry_Refresh`, the new pipeline that chains Silver → Gold → Headway every 15–30 min (re-select its three notebooks after Git sync). After the first run, check the quarantine breakdown:

```
SELECT failed_rules, event_type, COUNT(*) n FROM silver.telemetry_quarantine
GROUP BY failed_rules, event_type ORDER BY n DESC
-- expect only small counts (bad GPS etc.). A rule failing >1% of a
-- category means the RULE is wrong about the feed, not the data.
```

2. `NB_Gold_Deviation` — run TWICE; the row count must not change (ingest-time window + MERGE on `event_id`). Then `NB_Gold_Headway_Bunching`. Chain 1→2→3 on a 15–30 min schedule.

■ **Lesson:** Expected finding, not a bug: ~52% of stop events run early (schedule padding), ~2.6% late. TripUpdates carry no `vehicle_id` (DQ rule already scoped); vehicle positions between trips have no `trip_id` and quarantine as designed.

Phase 7 — Serving

KQL Real-Time Dashboard on `latest_positions` (live map, delay heat by line); Direct Lake semantic model + report over Gold (OTP trends, worst stops, bunching, schedule-version comparison); Activator rule on bunching events → Teams/email. Commit each new item to Git as soon as it exists.

Phase 8 — CI/CD promotion

1. Test/prod workspaces; service principal admin on both; tenant setting 'SPs can use Fabric APIs' enabled.
2. ADO: variable group `marta-pulse` (`serviceConnection`, `functionAppName`, `fabricWorkspaceIdTest/Prod`); environment `marta-pulse-prod` with approval; real GUIDs in `deploy/parameter.yml` + `deploy/config/*.yml`.
3. Register both pipelines; PR → CI (now also compile-checks the notebooks); merge → Function deploy → `fabric-cicd` to test → approval → prod. Upload the wheel to test/prod Environments too — `fabric-cicd` deploys items, not Environment libraries.
4. `TODO.md` carries the remaining deferred items: CD remote-build hardening, local-hour OTP aggregation.

Teardown order for the rebuild: delete the Fabric workspace items (Eventstream first, so nothing writes), delete and recreate the workspace if you want a clean slate, keep the ADO repo (it now holds everything). Delete and redeploy `rg-martapulse-dev` this time — the plan SKU change (Y1 → FC1) requires fresh resources.